实验 4. 回溯法的应用- 0-1 背包等问题

实验内容

本实验要求基于算法设计与分析的一般过程（即待求解问题的描述、算法设计、算法描述、算法正确性证明、算法分析、算法实现与测试），通过回溯法的在实际问题求解实践中，加深理解其基本原理和思想以及求解步骤。求解的问题为 0-1 背包。**作为挑战：可以考虑回溯法在如最大团、旅行商、图的 m 着色等问题中的应用。**

实验目的

- 理解回溯法的核心思想以及求解过程（确定解的形式及解空间组织，分析出搜索过程中的剪枝函数即约束函数与限界函数）；
- 掌握对几种解空间树（**子集树**、排列数、满 m 叉树）的回溯方法；
- 从算法分析与设计的角度，对 0-1 背包等问题的基于回溯法求解有进一步的理解。

环境要求

无特别要求。算法实现可以自由选择 C, C++, Java, Python 等。

实验步骤

步骤 1：理解问题，给出问题的描述。

步骤 2：算法设计，包括算法策略与数据结构的选择；

步骤 3：描述算法。采用源代码以外的形式（如**伪代码**、流程图等）来描述；

步骤 4：算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明（这里可以略去。有兴趣可以深入一下“多米诺性质”）；

步骤 5：算法复杂性分析，包括时间复杂性和空间复杂性（关于平均情况下的时间复杂性分析方法，有兴趣可以深入一下“蒙特卡洛方法”）；

步骤 6：算法实现与测试。附上代码或以附件的形式提交，同时贴上算法运行结果截图；

步骤 7：技术上、分析过程中等各种心得体会与备忘，需要言之有物。

说明：步骤 1-6 在“实验结果”一节中描述，步骤 7 在“实验总结”一节中描述。

实验结果

1、 理解问题，给出问题的描述。

n 个物品和 1 个背包。对物品 i，其价值为 vi，重量为 wi，背包容量为 W。如何选取物品装入背包，使背包中所装入的物品的总价值最大？其中，wi, W 都是正整数。

2、 算法设计，包括算法策略与数据结构的选择；

将问题的解表示为 n 元组(x1,x2,⋯,xn)的形式，则可知通过回溯法最多能产 2^n 种解，这些解构成了解空间，即子集树，在使用回溯法时，由于限界条件

cp+brp>bestp，许多解将会被剪枝，优化了算法，最后再通过 dfs 即可获得最优解。在构建解集树时，对每个结点的拓展即为对 xi 取值的选择(放入/不放入背包)，且判断是否符合约束条件与限界条件，当拓展到叶子结点后再回溯到之前的结点，判断是否还有可以拓展的结点，最终得到解集树。

3、 描述算法。希望采用源代码以外的形式，如**伪代码**、流程图等；

```
void backtrackKnapsack01(t)
    if t > n
        memcpy(bestx,x)
        bestp = nowV
        return
    if nowV+w[t] <= W
        x[t] = 1
        nowW += w[t]
        nowV += v[t]
        backtrackKnapsack01(t+1)
        nowW -= w[t]
        nowV -= v[t]
    if bound(t+1,W-nowW,nowV) > bestp
        x[t] = 0
        backtrackKnapsack01(t+1)

void bound(i,rw,nv)
    b = nv
    while i <= n and w[i] <= rw
        b += v[i]
        rw -= w[i++]
    if i <= n
        b += (float)v[i]/w[i]*rw
    return b
```

4、 算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明；

通过 dfs 逐步确定 n 元组每个变量的取值后可以得到一个问题的解，当对整个解空间搜索完毕后，通过比较各所得解，即可得到最优解，在搜索过程中被限界条件所剪枝的分枝，因为该分枝不是最优解，根据子集树的多米诺性质可知，由该分枝所得的问题的解也必然不是最优解，故可以剪去。

5、 算法复杂性分析，包括时间复杂性和空间复杂性；

时间复杂度：约束函数耗时最坏为$O(2^n)$, 限界函数耗时最坏为$O(n2^n)$

故时间复杂度为$O(n2^n)$

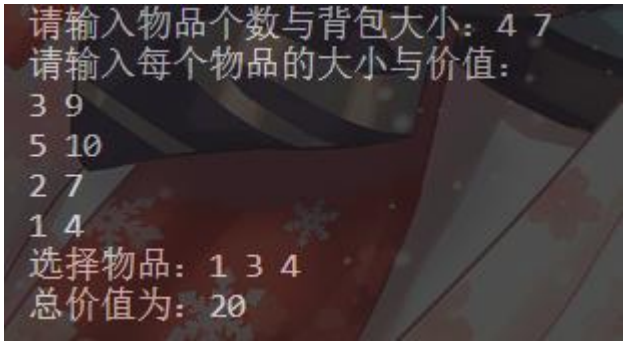
空间复杂度：由递归深度可知为$O(n)$

6、 算法实现与测试。附上代码或以附件的形式提交，同时贴上算法运行结果截图；

```
#include <iostream>
#include <algorithm>
using namespace std;
```

```
const int N = 1e3+10;

int w[N],v[N];
int W,n,nowW,nowV,bestp;
int *x,*bestx;

void backtrackKnapsack01(int);
void input();
void Print();
int bound(int, int, int);

int main(){

	input();
	backtrackKnapsack01(1);
	Print();
	delete[] x;
	delete[] bestx;
	return 0;

}

void input(){
	cout << "请输入物品个数与背包大小：";
	cin >> n >> W;
	x = (int*)new int[n];
	bestx = (int*)new int[n];
	cout << "请输入每个物品的大小与价值：" << endl;
	for (int i = 1; i <= n; i++) {
		cin >> w[i] >> v[i];
	}

}

void backtrackKnapsack01(int t){
	if(t>n){
		memcpy(bestx,x,(n+1)*sizeof(int));
		bestp = nowV;
		return;
	}
	if(nowW+w[t]<=W){
		x[t] = 1;
		nowW += w[t];
		nowV += v[t];
```

```
            backtrackKnapsack01(t+1);
            nowW -= w[t];
            nowV -= v[t];
        }
        if(bound(t+1,W-nowW,nowV) > bestp){
            x[t] = 0;
            backtrackKnapsack01(t+1);
        }
}

int bound(int i, int rw, int nv){
        int b = nv;
        while(i <= n && w[i] <= rw){
            b += v[i];
            rw -= w[i++];
        }
        if(i <= n){
            b += (float)v[i] / w[i] * rw;
        }
        return b;
}

void Print(){
        cout << "选择物品：";
        for (int i = 1;i <= n;i++){
            if(bestx[i]){
                cout << i << " ";
            }
        }
        cout << "\n" << "总价值为：" << bestp << endl;
}
```



实验总结

本次实验遇到的问题及其解决方法（如果有的话），以及心得体会等的总结。需言之有物。

在本次实验中，我进行了回溯法的具体应用，体会到了回溯法的优点，回溯法可以看作带剪枝的 dfs，思路较易想到，与穷举法相比，回溯法具有更优秀的时间复杂度，优化了穷举法。当能将问题的解空间定义出来时，使用回溯法可以比较方便的得出问题的解